

C Programming

An objective question bank

**Department of Computer Science & Engineering
Birla Institute of Science & Technology
Jaipur Campus**

Topics covered by this question bank:

1. Introduction and Basic concepts of C language

1. General Introduction
2. Console Input\Output
3. Operators
4. Conditions
5. Looping
6. Arrays

2. Functions

3. Pointers

4. Structures & Unions

5. File Handling

Q.1. Which of the following correctly declares an array?

- A. int an array[10];
- B. int an array;
- C. an array{10};
- D. array an array[10];

Q.2. What is the index number of the last element of an array with 29 elements?

- A. 29
- B. 28
- C. 0
- D. Programmer-defined

Q.3. Which of the following is a two-dimensional array?

- A. array anarray[20][20];
- B. int anarray[20][20];
- C. int array[20, 20];
- D. char array[20];

Q.4. What will be output if you will compile and execute the following c code?

```
#include<stdio.h>
int main(){
int i=320;
char *ptr=(char *)&i;
printf("%d",*ptr);
return 0;
}
```

- (A) 320
- (B) 1
- (C) 64
- (D) Compiler error
- (E) None of above

Q.5. What will be output if you will compile and execute the following c code?

```
#include<stdio.h>
#define x 5+2
```

```
int main(){
int i;
i=x*x*x;
printf("%d",i);
return 0;

}
```

- (A) 343
- (B) 27
- (C) 133
- (D) Compiler error
- (E) None of above

Q.6. What will be output if you will compile and execute the following c code?

```
#include<stdio.h>
int main(){
char c=125;
c=c+10;
printf("%d",c);
return 0;

}
```

- (A) 135
- (B) +INF
- (C) -121
- (D) -8
- (E) Compiler error

Q.7 What will be output if you will compile and execute the following c code?

```
#include<stdio.h>int main(){
float a=5.2;
if(a==5.2)
printf("Equal");
else if(a<5.2)
printf("Less than");
else
printf("Greater than");
return 0;
```

}

- (A) Equal
- (B) Less than
- (C) Greater than
- (D) Compiler error
- (E) None of above

Q.8 What will be output if you will compile and execute the following c code?

```
#include<stdio.h>
int main(){
int i=4,x;
x=++i + ++i + ++i;
printf("%d",x);
return 0;
}
```

- (A) 21
- (B) 18
- (C) 12
- (D) Compiler error
- (E) None of above

Q.9 What will be output if you will compile and execute the following c code?

```
#include<stdio.h>
int main(){
int a=2;
if(a==2){
a=~a+2<<1;
printf("%d",a);
}
else{
break;
}
return 0;
}
```

- (A) It will print nothing.
- (B) -3

- (C) -2
- (D) 1
- (E) Compiler error

Q.10.What will be output if you will compile and execute the following c code?

```
#include<stdio.h>
int main(){
int a=10;
printf("%d %d %d",a,a++,++a);
return 0;
}
```

- (A) 12 11 11
- (B) 12 10 10
- (C) 11 11 12
- (D) 10 10 12
- (E) Compiler error

Q.11.What will be output if you will compile and execute the following c code?

```
#include<stdio.h>
int main(){
char *str="Hello world";
printf("%d",printf("%s",str));
return 0;
}
```

- (A) 11Hello world
- (B) 10Hello world
- (C) Hello world10
- (D) Hello world11
- (E) Compiler error

Q.12.What will be output if you will compile and execute the following c code?

```
#include <stdio.h>
#include <string.h>
int main(){
char *str=NULL;
```

```
strcpy(str,"cquestionbank");  
printf("%s",str);  
return 0;  
}
```

- (A) cquestionbank
- (B) cquestionbank\0
- (C) (null)
- (D) It will print nothing

Q.13.What will be output if you will compile and execute the following c code?

```
#include <stdio.h>  
#include <string.h>  
int main(){  
int i=0;  
for(;i<=2;)  
printf(" %d",++i);  
return 0;  
}
```

- (A) 0 1 2
- (B) 0 1 2 3
- (C) 1 2 3
- (D) Compiler error
- (E) Infinite loop

Q.14.What will be output if you will compile and execute the following c code?

```
#include<stdio.h>  
int main(){  
int x;  
for(x=1;x<=5;x++);  
printf("%d",x);  
return 0;  
}
```

- (A) 4
- (B) 5
- (C) 6

- (D) Compiler error
- (E) None of above

Q.15.What will be output if you will compile and execute the following c code?

```
#include<stdio.h>
int main(){
printf("%d",sizeof(5.2));
return 0;

}
```

- (A) 2
 - (B) 4
 - (C) 8
 - (D) 10
 - (E) Compiler error
- 13.

Q.16.What will be output if you will compile and execute the following c code?

```
#include <stdio.h>
#include <string.h>
int main(){
char c='\08';
printf("%d",c);
return 0;

}
```

- (A) 8
- (B) "8"
- (C) 9
- (D) null
- (E) Compiler error

Q.17.What will be output if you will compile and execute the following c code?

```
#include<stdio.h>
#define call(x,y) x##y
int main(){
```



```
int x=5,y=10,xy=20;
printf("%d",xy+call(x,y));
return 0;

}
```

- A.35
- B.510
- C.15
- D.40
- E. None of above

Q.18.What will be output if you will compile and execute the following c code?

```
#include<stdio.h>
int * call();
int main(){
int *ptr;
ptr=call();
printf("%d",*ptr);
return 0;
```

```
int * call(){
int a=25;
a++;
return &a;
}
```

- (A) 25
- (B) 26
- (C) Any address
- (D) Garbage value
- (E) Compiler error

Q.19.
What is error in following declaration?

```
struct outer{
int a;
struct inner{
char c;
};
```

};

- (A) Nesting of structure is not allowed in c.
- (B) It is necessary to initialize the member variable.
- (C) Inner structure must have name.
- (D) Outer structure must have name.
- (E) There is not any error.

Q.20.What will be output if you will compile and execute the following c code?

```
#include<stdio.h>
int main(){
int array[]={10,20,30,40};
printf("%d",-2[array]);
return 0;

}
```

- (A) -60
- (B) -30
- (C) 60
- (D) Garbage value
- (E) Compiler error

Q.20 What will be output if you will compile and execute the following c code?

```
#include<stdio.h>
int main(){
int i=10;
static int x=i;
if(x==i)
printf("Equal");
else if(x>i)
printf("Greater than");
else
printf("Less than");return 0;

}
```

- (A) Equal

- (B) Greater than
- (C) Less than
- (D) Compiler error
- (E) None of above

Q.21.What will be output if you will compile and execute the following c code?

```
#include<stdio.h>
#define max 5;
int main(){
int i=0;
i=max++;
printf("%d",i++);
return 0;
```

```
}
```

- (A) 5
- (B) 6
- (C) 7
- (D) 0
- (E) Compiler error

Q.22

```
int z,x=5,y=-10,a=4,b=2;
```

```
z = x++ - --y * b / a;
```

What number will z in the sample code above contain?

- A. 5
- B. 6
- C. 10
- D. 11
- E. 12

Q.23

Code:

```
void *ptr;
```

```
myStruct myArray[10];
```

```
ptr = myArray;
```

Which of the following is the correct way to increment the variable "ptr"?

- A. ptr = ptr + sizeof(myStruct); [Ans]
- B. ++(int*)ptr;

- C. ptr = ptr + sizeof(myArray);
- D. increment(ptr);
- E. ptr = ptr + sizeof(ptr);

Q.24

Code:

```
char* myFunc (char *ptr)
{
    ptr += 3;
    return (ptr);
}
int main()
{
    char *x, *y;
    x = "HELLO";
    y = myFunc (x);
    printf ("y = %s \n", y);
    return 0;
}
```

What will print when the sample code above is executed?

- A. y = HELLO
- B. y = ELLO
- C. y = LLO
- D. y = LO
- E. x = O

Q.25

Code:

```
struct node *nPtr, *sPtr;    /* pointers for a linked list. */
for (nPtr=sPtr; nPtr; nPtr=nPtr->next)
{
    free(nPtr);
}
```

The sample code above releases memory from a linked list. Which of the choices below accurately describes how it will work?

- A. It will work correctly since the for loop covers the entire list.
- B. It may fail since each node "nPtr" is freed before its next address can be accessed.
- C. In the for loop, the assignment "nPtr=nPtr->next" should be changed to "nPtr=nPtr.next".

- D. This is invalid syntax for freeing memory.
- E. The loop will never end.

Q.26

What function will read a specified number of elements from a file?

- A. fileread()
- B. getline()
- C. readfile()
- D. fread()
- E. gets()

27

"My salary was increased by 15%!"

Select the statement which will EXACTLY reproduce the line of text above.

- A. printf("\nMy salary was increased by 15/%!\n");
- B. printf("My salary was increased by 15%\n");
- C. printf("My salary was increased by 15'!\n");
- D. printf("\nMy salary was increased by 15%%!\n");
- E. printf("\nMy salary was increased by 15%!\n");

Q.28

What is a difference between a declaration and a definition of a variable?

- A. Both can occur multiple times, but a declaration must occur first.
- B. There is no difference between them.
- C. A definition occurs once, but a declaration may occur many times.
- D. A declaration occurs once, but a definition may occur many times.
- E. Both can occur multiple times, but a definition must occur first.

Q.29

```
int testarray[3][2][2] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
```

What value does testarray[2][1][0] in the sample code above contain?

- A. 3
- B. 5
- C. 7
- D. 9
- E. 11

Q.30

Code:

```
int x[] = { 1, 4, 8, 5, 1, 4 };
```

```
int *ptr, y;
```

```
ptr = x + 4;
```

```
y = ptr - x;
```

What does y in the sample code above equal?

A. -3

B. 0

C. 4

D. 4 + sizeof(int)

E. 4 * sizeof(int

Q.31

Code:

```
void myFunc (int x)
```

```
{
```

```
    if (x > 0)
```

```
        myFunc(--x);
```

```
    printf("%d, ", x);
```

```
}
```

```
int main()
```

```
{
```

```
    myFunc(5);
```

```
    return 0;
```

```
}
```

What will the above sample code produce when executed?

A.

1, 2, 3, 4, 5, 5,

B.

4, 3, 2, 1, 0, 0,

C.

5, 4, 3, 2, 1, 0,

D.

0, 0, 1, 2, 3, 4,

E.

0, 1, 2, 3, 4, 5,

Q.32

$11 \wedge 5$

What does the operation shown above produce?

- A.1
- B.6
- C.8
- D.14
- E.15

Q.33

```
#define MAX_NUM 15
```

Referring to the sample above, what is MAX_NUM?

- A.
MAX_NUM is an integer variable.
- B.
MAX_NUM is a linker constant.
- C.
MAX_NUM is a precompiler constant.
- D.
MAX_NUM is a preprocessor macro.
- E.
MAX_NUM is an integer constant.

Q.34. Which one of the following will turn off buffering for stdout?

- A.
setbuf(stdout, FALSE);
- B.
setvbuf(stdout, NULL);
- C.
setbuf(stdout, NULL);
- D.
setvbuf(stdout, _IONBF);
- E.
setbuf(stdout, _IONBF);

Q.35

What is a proper method of opening a file for writing as binary file?

- A.
FILE *f = fwrite("test.bin", "b");
- B.
FILE *f = fopenb("test.bin", "w");
- C.
FILE *f = fopen("test.bin", "wb");
- D.

```
FILE *f = fwriteb( "test.bin" );  
E.  
FILE *f = fopen( "test.bin", "bw" );
```

Q.36 Which one of the following functions is the correct choice for moving blocks of binary data that are of arbitrary size and position in memory?

- A. memcpy()
- B. memset()
- C. strncpy()
- D. strcpy()
- E. memmove()

Q.37

```
int x = 2 * 3 + 4 * 5;
```

What value will x contain in the sample code above?

- A. 22
- B. 26
- C. 46
- D. 50
- E. 70

Q.38

Code:

```
void * array_dup (a, number, size)  
    const void * a;  
    size_t number;  
    size_t size;  
{  
    void * clone;  
    size_t bytes;  
    assert(a != NULL);  
    bytes = number * size;  
    clone = alloca(bytes);  
    if (!clone)  
        return clone;  
    memcpy(clone, a, bytes);  
    return clone;  
}
```


The function `array_dup()`, defined above, contains an error. Which one of the following correctly analyzes it?

- A. If the arguments to `memcpy()` refer to overlapping regions, the destination buffer will be subject to memory corruption.
- B. `array_dup()` declares its first parameter to be a pointer, when the actual argument will be an array.
- C. The memory obtained from `alloca()` is not valid in the context of the caller. Moreover, `alloca()` is nonstandard.
- D. `size_t` is not a Standard C defined type, and may not be known to the compiler.
- E. The definition of `array_dup()` is unusual. Functions cannot be defined using this syntax.

Q.39

```
int var1;
```

If a variable has been declared with file scope, as above, can it safely be accessed globally from another file?

- A. Yes; it can be referenced through the register specifier.
- B. No; it would have to have been initially declared as a static variable.
- C. No; it would need to have been initially declared using the global keyword .
- D. Yes; it can be referenced through the publish specifier.
- E. Yes; it can be referenced through the extern specifier.

Q.40

```
time_t t;
```

Which one of the following statements will properly initialize the variable `t` with the current time from the sample above?

- A. `t = clock();`
- B. `time( &t );`
- C. `t = ctime();`
- D. `t = localtime();`
- E. None of the above

Q.41 Which one of the following provides conceptual support for function

calls?

- A. The system stack
- B. The data segment
- C. The processor's registers
- D. The text segment
- E. The heap

Q.42 C is which kind of language?

- A. Machine
- B. Procedural
- C. Assembly
- D. Object-oriented
- E. Strictly-typed

Q.43

Code:

```
int i,j;
int ctr = 0;
int myArray[2][3];
for (i=0; i<3; i++)
    for (j=0; j<2; j++)
    {
        myArray[j][i] = ctr;
        ++ctr;
    }
```

What is the value of myArray[1][2]; in the sample code above?

- A.1
- B.2
- C.3
- D.4
- E.5

Q.44

Code:

```
int x = 0;
for (x=1; x<4; x++);
printf("x=%d\n", x);
```

What will be printed when the sample code above is executed?

- A. x=0
- B. x=1
- C. x=3
- D. x=4
- E. x=5

Q.45

Code:

```
int x = 3;  
if( x == 2 );  
    x = 0;  
if( x == 3 )  
    x++;  
else x += 2;
```

What value will x contain when the sample code above is executed?

- A.1
- B.2
- C.3
- D.4
- E.5

Q. 46 . TurboC 3.0 is based on

- a . DOS
- b . UNIX
- c . Windows
- d . none

Q. 47 . Find the output.

```
void main()  
{  
    int a=4,b=5;  
    printit(a, b);  
}  
printit(int b, int a)  
{  
    printf("%d %d", a, b);  
}  
int a=0;  
int b=1;  
printf("%d %d", a, b);  
}  
}
```

- a . 4 5 0 1
- b . 5 4 0 1
- c . 5 4 5 4
- d . None of these

Q.48

```
printf ("%d", printf ("tim"));
```

- (A) results in a syntax error
- (B) outputs tim3
- (C) outputs garbage
- (D) outputs tim and terminates

Q. 49 . The contents of a file will be lost if it is opened in

- a . a mode
- b . a- mode
- c . w+ mode
- d . a+ mode

Q. 29 . Find the output.

```
void main()
{
int i=1,j=2,k=3;
if(i==1)
if(j==2)
if(k==3)
{
printf("ok");
break;
}
else
printf("continue");
printf("bye");
}
```

- a . ok
- b . okbye
- c . Misplaced break
- d . None of these

Q. 30 . Find the output

```
void main()
{
printf("Lakshya\C\Academy");
}
```

- a . LakshyaCAcademy
- b . LakshyaAcademy

c . Lakshya
d . Lakshyacademy

Q. 50 . Find out the output

```
void main()
{
int val=1234;
int *ptr=&val;
printf("%d %d",++val,(*(int *)ptr)--);
}
```

- a . 1234 1233
- b . 1235 1234
- c . 1234 1234
- d . None of these

Q. 51 . Find the correct output

```
void main()
{
int i,j=6;
for(;i=j;j-=2)
printf("%d",j);
}
```

- a . Error
- b . Garbage values
- c . 642
- d . 6420

Q. 52 . State the correct statement.

- a . In a while loop the control conditional check is performed n times.
- b . In a do-while loop the control conditional check is performed n+1 times.
- c . Break is a keyword used with if and switch case.
- d . None of these.

Q. 53 . Find out the output

```
void main()
{
printf("3", "%f", 4);
}
```

- a . 34.000000
- b . 4
- c . 3
- d . Error

Q. 54 . What happens when subscript used for an array exceeds the array size?

- a . Compiler gives an error message
- b . Simply placed excess data outside the array
- c . Simply placed excess data on top of other data or on the program itself.
- d . The program must be compiled.

Q. 55 . The order of evaluation of operators having same precedence is decided by

- a . Associativity rule
- b . Precedence rule
- c . Left-right rule
- d . Right-left rule

Q. 56 . The set of routines stored in ROM that enable a computer to start the O.S and to communicate with the various devices in the system is known as

- a . device driver
- b . boot parameter
- c . bios
- d . none of these

Q. 57 . Find the output

```
struct main
```

```
{
```

```
int n;
```

```
int (*p)();
```

```
};
```

```
void main()
```

```
{
```

```
struct main m;
```

```
int fun();
```

```
m.p=fun();
```

```
*(m.p);
```

```
}
```

```
fun()
```

```
{
```

```
printf("Hallo");
```

```
}
```

```
a . Hallo
```

- b . HalloHallo
- c . Error
- d . No output

Q. 58 . Find the output

```
union uni
{
int x;
int a:8;
char b:8;
}
void main()
{
union uni u={48};
printf("%d %C", u.a,u.b);
}
```

- a . Invalid initialization
- b . 48 0
- c . 48 48
- d . 0 0

Q. 59 . An AND gate

- a . implements logical addition
- b . is equivalent to a series switching circuit
- c . implements logical subtractions
- d . is equivalent to a parallel switching circuit

Q. 23 . Find the output

```
void main()
{
goto cite;
{
static int a=10;
cite: printf("%d",a);
}
}
```

- a . 10
- b . Warning, unreachable code
- c . Error
- d . Garbage value

Q. 60 . Find out the output

```
void main()
{
```

```
int i=3,j=2,k=1;  
printf("%d / %d");  
}
```

- a . 1
- b . 0
- c . 3/2
- d . 1/2

Q. 25 . The 'continue' statement is used to

- a . Continue the next iteration of a loop construct.
- b . Exit the block where it exists and continues after.
- c . Exit the outermost block even if it occurs inside the innermost.
- d . Continue the compilation even an error occurs in a program.

Q. 61 . which of the following declaration is incomplete type?

- (1) struct tag;
- (2) struct tag {int a };
- (3) typedef struct{int a;}abc

- a . 1 only
- b . 1 and 2 only
- c . 1 and 3 only
- d . 2 and 3 only

Q. 62 . Find the odd one out

- a . #elif
- b . #line
- c . #else
- d . #ifdef

Q. 63 . The 'continue' statement is used to

- a . Continue the next iteration of a loop construct.
- b . Exit the block where it exists and continues after.
- c . Exit the outermost block even if it occurs inside the innermost.
- d . Continue the compilation even an error occurs in a program.

Q. 64 . What is the value of EOF which is declared in "stdio.h"?

- a . 0
- b . -1
- c . Any positive value
- d . None of these

Q. 65 . Which data type behaves like both integer type and character

- type?
- a . short int
 - b . signed int
 - c . char
 - d . enum

Q. 66 . Find the correct output

```
#include "string.h"
void main()
{
    struct node
    {
        int data;
        struct node *next;
    };
    struct node *p, *q;
    p=(struct node *)malloc(sizeof(struct node));
    q=(struct node *)malloc(sizeof(struct node));
    p->data=10;
    q->data=20;
    p->next=q;
    q->next=NULL;
    printf("%d",p->data);
    p=p->next;
    printf("%d",p->data);
}
a . 10 20
b . 10 Garbage value
c . 20 Garbage value
d . None of these
```

Q. 67 . Find out the output

```
void main()
{
    int x=2,*y=&x;
    printf("%d",x*y);
}
a . 4
b . Garbage
c . Prints result of multiplication of address of x and x
d . Error
```

Q. 68 . Which of the following function is appropriate to read one

character at a time?

- a . fscanf()
- b . fgetc()
- c . read()
- d . fgets()

Q. 69 . Find out the output

```
main()
{
const int x=get();
printf("%d",x);
}
get()
{
return(20);
}
a . 20
b . Garbage value
c . Error
d . 0
```

Q. 70 . Which format specifier is used to find out the offset address of a variable?

- a . %o
- b . %p
- c . %h
- d . %d

Q. 6 . Find out the output.

```
#define fool !1 - -1
void main()
{
printf("Hi!");
if (fool)
printf("Bye");
}
a . Lvalue required
b . Hi!
c . Bye
d . Hi!Bye
```

Q. 71 . Find out the output

```
void main()
{
```

```

char str[20];
static int i;
for(;;)
{
i++[str]= 'a'+15;
if(i==19)
break;
i++;
}
i[str]='\0';
printf("%s",str);
}

```

- a . Error
- b . Garbage value
- c . p
- d . ppppppppppppppppppppppppp

Q. 72 . When loop loses its stop value then it is called

- I. Odd loop
 - II. Even loop
 - III. Unknown loop
 - IV. User friendly loop
- a . only I
 - b . I & III
 - c . III & IV
 - d . II & IV

Q. 73 . Find the correct output

```

void main()
{
int x=10,y=20,p,q;
p=add(x,y);
q=add(x,y);
printf("%d %d",p,q);
}
add(int a,int b)
{
a+=a;
b+=b;
return(a);
return(b);
}

```

- a . 10 10
- b . 20 20

- c . 10 20
- d . 20 10

Q. 74 . Find out the output

```
main()
{
int a[5] = {1,2,3,4,5};
int *ptr = (int*)(&a+1);
printf("%d %d" ,*(a+1),*(ptr-1));
}
```

- a . 2 2
- b . 2 1
- c . 2 5
- d . None of the above

Q. 75 . The area or scope of the variable depends on its

- a . Date type
- b . Storage class
- c . System type
- d . None of these

Q. 12 . Each case statement in switch is separated by

- a . break
- b . continue
- c . exit()
- d . goto

Q. 76 . Find out the output

```
main()
{
const int x=get();
printf("%d",x);
}

get()
{
return(20);
}
```

- a . 20
- b . Garbage value
- c . Error
- d . 0

Q. 77 . Find out the output

```
void main()
{
```

```

int a[5]={1,2,3};
int j;
for(j=0;j<5;j++) x1="5.74,x2=" x1="5.74" x2="23.78" opname =" Xstr(OP);"
p="c;" p="lakshya" x="10;" x="callme(x);" i="5;" x="x/2-3;" s=""\dx';"
i="2;">>1)
{
default: i++;
case 1: ;
case 2: ;
}
printf("%d",i);
}

```

- a . 2
- b . 1
- c . Expression syntax error
- d . None of these

Q. 78 . Find the correct output

```

void main()
{
int a,b,c;
c=scanf("%d%d",&a,&b);
printf("\n%d",c);
}

```

- a . 1
- b . 2
- c . 0
- d . None of these

Q. 79 . Which of the following is known as auxillary memory?

- a . RAM
- b . ROM
- c . HARD DISK
- d . BIOS

Q. 80 . Choose the wrong one

- a . A structure can be nested within same structure
- b . A value of one structure variable can be assigned to another structure variable of same or different type.

- c . It is illegal to use the structure itself as its member.
- d . In self referential structure one member must be a pointer type.

Q. 81 . Find out the output

```
void main()
{
    register int x;
    scanf("%d",&x);
    printf("%d",x);
}
```

If the value x is given 5 then output will be

- a . 5
- b . 0
- c . Garbage
- d . Error

Q. 82 . Find the output

```
#define float int
void main()
{
    int x=10;
    float y=3;
    y=x%y;
    printf("%f",y);
}
```

- a . 0
- b . 1
- c . 1.000000
- d . Error, floating point format not linked

Q. 83. Which of the following is a miscellaneous directive?

- a . #undef
- b . #error
- c . #elif
- d . #include

Q. 84 . State the correct statement.

- a . Linker combines different source files before compilation.
- b . Linker combines different object files before execution.
- c . Linker expands different source files before compilation.
- d . none of these

Q. 85 . Find out the output.

```
void main()
```

```
{
int n=5;
if(n==5?printf("Hallo):(printf("Hai"),printf("Bye"));
}
```

a . HalloBye
b . Hallo
c . Expression syntax error
d . HaiBye

Q. 86 . Find the output

```
int x,y;
void main()
{
show(&x,&y);
printf("%d %d",x,y);
}
show(int *x,int *y)
{
*x++;
*y++;
}
```

a . 0 0
b . 1 1
c . Garbage output
d . None of these

Q. 87 . The different models used to organize data in the secondary memory are collectively called as

- a . File structures
- b . Data structures
- c . Self referential structures
- d . None of these

Q. 88 . Find the output

```
void main()
{
printf("%f",(float)9/5);
}
```

a . 1.8
b . 1.0
c . 2.0
d . None of these

Q. 89 . Find out the output

```
void main()
{
int static auto x;
x=5;
printf("%d",++x);
x--;
printf("%d",x);
}
```

- a . Too many storage classes in declaration
- b . 6 5
- c . 6 6
- d . None of these

Q. 90 . Find the output.

```
void main()
{
int x=5;
char a[x]={‘a’,’b’,’d’};
printf("%c",++1[a]);
}
```

- a . b
- b . c
- c . d
- d . None of these

Q. 91 . Addresses in Memory always stored in ABCD order but data is stored in

- a . ABCD order
- b . DCBA order
- c . Depends on system type
- d . None of these

Q. 13 . A bit field can be of

- a . int
- b . float
- c . double
- d . All of these

Q. 14 . The synonym for an existing data type can be created using

- a . typedef
- b . structure
- c . enum
- d . All of the above

Q. 92 . Which of the following is best suited for memory optimization

- a . Union

- b . Self referential structure
- c . Array
- d . Linked list

Q. 93 . Find out the output

```
#include"fcntl.h"
#include"sys\stat.h"
void main()
{
int x;
x=open("raja.txt",O_CREATO_TEXT, S_IWRITE);
printf("%d",x);
}
```

- a . 5
- b . 6
- c . 7
- d . None of these

Q. 94 . All structure variables are created in

- a . Code area
- b . Stack area
- c . Uninitialized data area
- d . Initialized data area

Q. 95 . Cast operator returns

- a . Lvalue
- b . Rvalue
- c . Integral value
- d . None

Q. 96 . Find the correct output

```
#define div(x) x/x
void main()
{
printf("%d",div(16)*16);
}
```

- a . 1
- b . 16
- c . 256
- d . None of these

Q. 97 . What is the formula to find real address from 16-bit physical address?

- a . segment address*16+offset address

- b . segment address+offset address *16
- c . segment address*10+offset address
- d . segment address+offset address *10

Q. 98. Which of the following function is appropriate to read one character at a time?

- a . fscanf()
- b . fgetc()
- c . read()
- d . fgets()

Q. 99 . Find the correct output

```
void main()
{
int a;
char b;
float c;
printf("%d",sizeof(a+sizeof(b+c)));
}
a . 2
b . 1
c . 4
d . Error
```

Q. 100 . What is the formula to find real address from 16-bit physical address?

- a . segment address*16+offset address
- b . segment address+offset address *16
- c . segment address*10+offset address
- d . segment address+offset address *10

Q. 101 . Which of the following function is appropriate to read one character at a time?

- a . fscanf()
- b . fgetc()
- c . read()
- d . fgets()

Q. 102 . Find the correct output

```
void main()
{
int a;
char b;
float c;
```

```
printf("%d",sizeof(a+sizeof(b+c)));  
}  
a . 2  
b . 1  
c . 4  
d . Error
```

Q. 103 . Find the correct output

```
1. #define p 2*2  
2. void main()  
3. {  
4. #define p 4*4  
5. printf("%d",p);
```

a . Error: In line no.1
b . Error: In line no.4
c . 4
d . 16

Q. 104 . Find the correct output

```
void main()  
{  
int p=7,q=9;  
p=p^q;  
q=q^p;  
printf("%d %d",p,q);  
}
```

a . 7 9
b . 9 7
c . 18 9
d . 14 7

Q. 105 . which of the following is a Manifest?

a . #define Lakshya 15
b . #defin Lakshya 15
c . #define Lakshya =15
d . #define Lakshya 15;

Q. 106 . Find out the output

```
void main()  
{  
float x=2.8,y=4;  
if(x%=y)  
printf("Both are equal");
```

```
else
printf("Not equal");
}
a . Both are equal
b . Not equal
c . Error
d . None of these
```

Q. 107 . Find the correct output

```
void main()
{
int a=2,b=0,c=-2;
if(b,a,c)
printf("Lack of C knowledge");
else
printf("Beginner");
}
a . Lack of C knowledge
b . Beginner
c . Compile time error
d . Run time error
```

Q. 108 . Find the correct output

```
#define SWAP(type,i,j) {type t=i;i=j;j=t;}
void main()
{
int s=5,t=2;
SWAP(int,s,t);
printf("%d %d",s,t);
}
a . 5 2
b . 2 5
c . 5 5
d . None
```

Q. 109 . Find the correct output

```
void main()
{
struct stu
{
int roll;
char name[10];
};
struct stu s={10,"Tuni"};
```

```

call(s);
}
call(struct stu s)
{
printf("%d %s",s.roll,s.name);
}

```

- a . 10 Tuni
- b . Compilation Error
- c . Runtime Error
- d . None of these

Q. 110 . Find the o/p

```

void main()
{
int x=5,y=3;
if(x>y)
printf("HOT");
else;
printf("COLD");
}

```

- a . HOT
- b . COLD
- c . HOTCOLD
- d . Compilation error

Q. 111 . The main function of shared memory is to

- a . use primary memory efficiently.
- b . do inter process communication.
- c . do intra process communication.
- d . none of these

Q. 112 . Switch case is very useful in

- a . Yes-no problem
- b . Menu-driven program
- c . Game programming
- d . All the above

Q. 113 . Find out the output.

```

#define PR(x) printf("%d ",x);
#define NULL printf("error");
#define PRINT(x1,x2) PR(x1); PR(x2); NULL
void main()
{

```

```
int x=5,y=10;
PRINT(x,y);
}
```

a . 5 10 error
b . 10 5 error
c . 5 10
d . 10 5

Q. 114 . Find the output

```
#define MAX(x,y) x>y?x:y
void main()
{
int x=2,y=3;
printf("%d",MAX(++x,++y));
}
```

a . 4
b . 5
c . Error
d . None of these

Q. 115 . Find out the output

```
void main()
{
unsigned int a=65536;
if(!a)
printf("%d",a,++a);
else
printf("%d",a,++a);
}
```

a . 0
b . 1
c . 65536
d . 65537

Q. 116 . $x\%y$ is equal to

a . $y-(x/y)*x$
b . $x-(x/y)*y$
c . $x-(y/x)*x$
d . $y-(y/x)*y$

Q. 117 . #define dprintf(e) printf(#e " = %d\n",'e')

```
void main()
{
int x=8;
```

```
int y=3;
dprintf(x/y);
}
a . expr=2
b . expr=101
c . x/y=101
d . None of the Above.
```

Q. 118 . Find out the output

```
void main()
{
int x=6,y=4,z=0;
if((x>=y)(z=y))
z-=2;
printf("%d",z);
}
a . 0
b . 2
c . -2
d . None of these
```

Q. 119 . Self referential structure

- a . Used to create a memory link.
- b . Plays an important role in linked list creation.
- c . Contains the object of same structure which is of pointer type.
- d . All the above

Q. 120 . Find out the output

```
typedef struct boy
{
float height;
char name:9;
int age:16;
}boy;
void main()
{
boy b={5.8,"Praveen kumar",24};
printf("%f %s %d",b.height,b.name,b.age);
}
a . 5.8 Praveen k 24
b . 5.8 Praveen kumar 24
c . 5.8 Praveen k 16
d . None of these
```

Q. 121 . Switch-case statement does not implement on

- a . non exclusive case
- b . mutually exclusive case
- c . mutually non exclusive case
- d . exclusive case

Q. 122 . Find out the output

```
void main()
{
    unsigned int i=2;
    i=((i<6)-1)^127; i="15;">y? x:y
    void main()
    {
```

- ```
 int a,b;
 a=MAX(1,2)+3;
 b=MAX(4,3)+5;
 printf("%d %d",a,b);
 }
 a . 5 4
 b . 5 9
 c . 5 8
 d . None of these
```

Q. 123 . Find the output

```
void main()
{
 (char)65='a';
 printf ("%c",*(char*)65);
}
```

- a . a
- b . A
- c . Error
- d . None of these

Q. 25 . The function ftell(fp) returns

- a . The beginning position of the file represented by fp.
- b . The end position of the file represented by fp.
- c . The current position of the file represented by fp.
- d . None of these

Q. 124 . Find out the output

```
void paws(int *apple,int *ball)
{
 *apple=*apple+*ball;
```



```

*ball=*apple-*ball;
*apple=*apple-*ball;
}
void main()
{
int apple=3617283450;
int ball=3617283449;
paws(&apple,&ball);
printf("%d %d",apple,ball);
}
a . 23929 23930
b . 23322 23323
c . 23939 23940
d . Error

```

Q. 125 . Find the correct output

```

void main()
{
char a[4]="rama";
char b[]="shyama";
printf("%d %d", sizeof(a),sizeof(b)) ;
}
a . 4 7
b . 5 6
c . 5 7
d . 4 6

```

Q. 126 . Find the correct output

```

typedef struct tag
{
int i;
char c;
}tag;
void main()
{
struct tag t1={1,'C'};
tag t2={2,'A'};
printf("%d%c ",t1.i,t1.c);
printf("%d%c ",t2.i,t2.c);
}
a . Error in first printf() statement.
b . Error in second printf() statement.
c . 1C 2A
d . None of these

```

Q. 127 . Find the correct output

```
void main()
{
 int i=5;
 i=++i + ++i + i++;
 printf("%d",i);
}
```

a . 21

b . 22

c . 19

d . None of these

Q. 30 . What will be the value of 'x' after execution of the following program?

```
void main()
{
 int x=!0*20;
}
```

a . 10

b . 20

c . 1

d . 0

Q. 128 . Find the correct output

```
void main()
{
 void show();
 show();
}

void show()
{
 char c[]="Lakshya\0\0";
 int i,*p;
 p=c;
 for(i=0;*p;i++)
 printf("%c",*p++);
}
```

a . Lakshya0

b . Lkha

c . Lkha0

d . Error

Q. 129 . Find the output

```
void main()
{
int x=2,y=3,z;
z=x++y++&&x++;
printf("%d %d %d",x,y,z);
}
```

a . 3 3 1  
b . 4 4 1  
c . 4 3 1  
d . None of these

Q. 130 . An I/O stream is

a . a text stream  
b . a binary stream  
c . Both a and b  
d . None of these

Q. 131 . Find out the output

```
void main()
{
int k=2,j=3,p=0;
p=(k,j,k);
k=p;
j=k;
printf("%d\n",p);
}
```

a . 2  
b . Error  
c . 0  
d . 3

Q. 132 . Find the output.

```
void main()
{
int x=5,y=6;
change(&x, &y);
printf("%d %d", x, y);
}
change(int *x, int *y)
{
int temp=1;
temp^=*x;
*x^=*y;
*y^=temp;
```

- }
- a . 5 6
- b . 6 5
- c . 3 2
- d . None of these

Q. 133 . Which of the following is an optical ROM?

- a . ROM
- b . PROM
- c . EPROM
- d . CDROM

Q. 134 . Find out the output

```
void e(int);
main()
{
int a;
a=3;
e(a);
}
void e(int n)
{
if(n>0)
{
e(--n);
printf("%d" , n);
e(--n);
}
}
a . 0 1 2 0
b . 0 1 2 1
c . 1 2 0 1
d . 1 2 0 1
```

Q. 135 . Find the output.

```
void main()
{
int array[]={1,3,5,7,9},*p,i;
p=array[0];
for(i=0;i<5;i++) c="65;" c="(!=" c="scanf(" y="Academy" i="3;i">=1;i--)
for(j=i;j>=1;j--)
printf("%d",i);
}
a . 321211
```

- b . 333221
- c . 332211
- d . 123233

Q. 136 . Find the incorrect one for 'typedef' storage class

- a . Permits descriptive names for datatypes.
- b . Renaming existing datatype
- c . Modification of the program is easier when host machine is changed.
- d . All of the above

Q. 137 . Find the output

```
void main()
{
printf("%d %d",sizeof('a'), sizeof('a1')) ;
}
```

- a . 1 2
- b . 1 1
- c . 2 1
- d . None of these

Q. 138 . Find the correct output

```
void main()
{
int a=2,b=4,c;
c=a>b?printf("2>=4):(puts("2<4");>=4
b . 2<4 a="100;" i="0;i<3;i++);" x="5;">
```

```
void main()
{
unsigned int a=-1;
printf("%d",~1);
}
```

- a . -3
- b . -1
- c . -2
- d . 1

Q. 139 . Which of the following is/are numeric type constants?

- a . integer constant and character constant
- b . integer constant and floating-point constant
- c . character constant and floating-point constant
- d . character constant and string constant

Q. 140 . Find the correct output

```
void main()
```

```

{
struct emp
{
int eid;
char ename[20];
};
struct emp e={010,"Deepak"};
show(s);
}
show(struct{int eid;char ename[20];}e)
{
printf("%d %s",e.roll,e.name);
}
a . Compilation Error
b . 010
c . 010 Deepak
d . None of these

```

Q. 141 . void main()

```

{
int i,j;
for(i=0,j=0;i<2,j<2;i++,j++) i="5;" a="1,b=" a==" b==" x="2,y=" t="x;"
x="*y;" y="t;" i="9;" i="0;i<3;i++)" x="3.7;" y="1.5;">1.5)
printf("fool");
else
printf("stupid");
}
a . fool
b . stupid
c . garbage value
d . None of these

```

Q. 142 . Most of the errors blamed on a computer are actually due to

- a . programming errors
- b . hardware damage
- c . data entry errors
- d . physical conditions

Q. 143 . Find the output.

```

void main()
{
int i;
for(i=-9;!i;i++);

```

```
printf("%d",-i);
}
a . 8
b . 9
c . 0
d . 1
```

Q. 144 . Switch-case statement does not implement on

- a . non exclusive case
- b . mutually exclusive case
- c . mutually non exclusive case
- d . exclusive case

Q. 145 . Find out the output.

```
void main()
{
int i=98;
while(100-i++)
printf("%u",i);
switch(i)
case 'e':
printf("Fool");
}
a . 98 99 100
b . 99 100
c . 99 100 Fool
d . Expression syntax error
```

Q. 146 . Find out the output

```
void main()
{
if("lakshya"=="lakshya")
printf("equal");
else
printf("not equal");
}
a . equal
b . not equal
c . Compile time error
d . None of these
```

Q. 147 . Find the output.

```
void main()
{
```

```
char s='A';
char a='a';
printf("%d",s-a);
}
```

a . -32  
b . -18  
c . Error  
d . None of these

Q. 148 . Find out the true statement.

- a . A union variable can be assigned to another union variable.
- b . The address of union variable can be extracted by using & operator.
- c . A function can return a pointer to the union.
- d . All the above

Q.149

Code:

```
char *ptr;
char myString[] = "abcdefg";
ptr = myString;
ptr += 5;
```

What string does ptr point to in the sample code above?

- A.fg
- B.efg
- C.defg
- D.cdefg
- E.None of the above

Q.150

Code:

```
double x = -3.5, y = 3.5;
printf("%.0f : %.0f\n", ceil(x), ceil(y));
printf("%.0f : %.0f\n", floor(x), floor(y));
```

What will the code above print when executed?  
ceil => rounds up 3.2=4 floor => rounds down 3.2=3

- A.-3 : 4  
-4 : 3
- B.-4 : 4  
-3 : 3
- C.-4 : 3  
-4 : 3
- D.-4 : 3  
-3 : 4



E -3 : 3  
-4 : 4

Q.151

Which one of the following will declare a pointer to an integer at address 0x200 in memory?

- A.int \*x;  
\*x = 0x200;
- B.int \*x = &0x200;
- C.int \*x = \*0x200;
- D.int \*x = 0x200;
- E.int \*x( &0x200 );

Q.152

Code:

```
int x = 5;
int y = 2;
char op = '*';
switch (op)
{
 default : x += 1;
 case '+' : x += y; /*It will go to all the cases*/
 case '-' : x -= y;
}
```

After the sample code above has been executed, what value will the variable x contain?

- A.4
- B.5
- C.6
- D.7
- E.8

Q.153

Code:

```
x = 3, counter = 0;
while ((x-1))
{
 ++counter;
 x--;
}
```

Referring to the sample code above, what value will the variable counter have when completed?

- A.0

- B.1
- C.2
- D.3
- E.4

Q.154

`char ** array [12][12][12];`

Consider array, defined above. Which one of the following definitions and initializations of p is valid?

- A. `char ** (* p) [12][12] = array;`
- B. `char ***** p = array;`
- C. `char * (* p) [12][12][12] = array;`
- D. `const char ** p [12][12][12] = array;`
- E. `char (** p) [12][12] = array;`

Q.155

`void (*signal(int sig, void (*handler) (int))) (int);`

Which one of the following definitions of `sighandler_t` allows the above declaration to be rewritten as follows:

`sighandler_t signal (int sig, sighandler_t handler);`

- A. `typedef void (*sighandler_t) (int);`
- B. `typedef sighandler_t void (*) (int);`
- C. `typedef void *sighandler_t (int);`
- D. `#define sighandler_t(x) void (*x) (int)`
- E. `#define sighandler_t void (*) (int)`

Q.156 All of the following choices represent syntactically correct function definitions. Which one of the following represents a semantically legal function definition in Standard C?

A.

Code:

```
int count_digits (const char * buf) {
 assert(buf != NULL);
 int cnt = 0, i;
 for (i = 0; buf[i] != '\0'; i++)
 if (isdigit(buf[i]))
 cnt++;
 return cnt;
}
```

B.

Code:

```
int count_digits (const char * buf) {
 int cnt = 0;
 assert(buf != NULL);
 for (int i = 0; buf[i] != '\0'; i++)
 if (isdigit(buf[i]))
 cnt++;
 return cnt;
}
```

C.

Code:

```
int count_digits (const char * buf) {
 int cnt = 0, i;
 assert(buf != NULL);
 for (i = 0; buf[i] != '\0'; i++)
 if (isdigit(buf[i]))
 cnt++;
 return cnt;
}
```

D.

Code:

```
int count_digits (const char * buf) {
 assert(buf != NULL);
 for (i = 0; buf[i] != '\0'; i++)
 if (isdigit(buf[i]))
 cnt++;
 return cnt;
}
```

E.

Code:

```
int count_digits (const char * buf) {
 assert(buf != NULL);
 int cnt = 0;
 for (int i = 0; buf[i] != '\0'; i++)
 if (isdigit(buf[i]))
 cnt++;
 return cnt;
}
```

Q.157

struct customer \*ptr = malloc( sizeof( struct customer ) );

Given the sample allocation for the pointer "ptr" found above, which one

of the following statements is used to reallocate ptr to be an array of 10 elements?

A.

`ptr = realloc( ptr, 10 * sizeof( struct customer ));`

B.

`realloc( ptr, 9 * sizeof( struct customer ) );`

C.

`ptr += malloc( 9 * sizeof( struct customer ) );`

D.

`ptr = realloc( ptr, 9 * sizeof( struct customer ) );`

E.

`realloc( ptr, 10 * sizeof( struct customer ) );`

Q.158 Which one of the following is a true statement about pointers?

A. Pointer arithmetic is permitted on pointers of any type.

B. A pointer of type `void *` can be used to directly examine or modify an object of any type.

C. Standard C mandates a minimum of four levels of indirection accessible through a pointer.

D. A C program knows the types of its pointers and indirectly referenced data items at runtime.

E. Pointers may be used to simulate call-by-reference.

Q.159 Which one of the following functions returns the string representation from a pointer to a `time_t` value?

A. `localtime`

B. `gmtime`

C. `strtime`

D. `asctime`

E. `ctime`

Q.160

Code:

```
short testarray[4][3] = { {1}, {2, 3}, {4, 5, 6} };
```

```
printf("%d\n", sizeof(testarray));
```

Assuming a short is two bytes long, what will be printed by the above code?

A. It will not compile because not enough initializers are given.

B. 6

C. 7

D. 12

E. 24

Q.161

```
char buf []= "Hello world!";
```

```
char * buf = "Hello world!";
```

In terms of code generation, how do the two definitions of buf, both presented above, differ?

A.The first definition certainly allows the contents of buf to be safely modified at runtime; the second definition does not.

B.The first definition is not suitable for usage as an argument to a function call; the second definition is.

C.The first definition is not legal because it does not indicate the size of the array to be allocated; the second definition is legal.

D.They do not differ -- they are functionally equivalent.

E.The first definition does not allocate enough space for a terminating NUL-character, nor does it append one; the second definition does.

Q.162

In a C expression, how is a logical AND represented?

A. @@

B. ||

C.AND.

D.&&

E.AND

Q.163

How do printf()'s format specifiers %e and %f differ in their treatment of floating-point numbers?

A. %e always displays an argument of type double in engineering notation; %f always displays an argument of type double in decimal notation. [Ans]

B. %e expects a corresponding argument of type double; %f expects a corresponding argument of type float.

C. %e displays a double in engineering notation if the number is very small or very large. Otherwise, it behaves like %f and displays the number in decimal notation.

D. %e displays an argument of type double with trailing zeros; %f never displays trailing zeros.

E. %e and %f both expect a corresponding argument of type double and format it identically. %e is left over from K&R C; Standard C prefers %f for new code.

Q.164

Which one of the following Standard C functions can be used to reset end-of-file and error conditions on an open stream?

- A. clearerr()
- B. fseek()
- C. ferror()
- D. feof()
- E. setvbuf()

Q.165

Which one of the following will read a character from the keyboard and will store it in the variable c?

- A. c = getc();
- B. getc( &c );
- C. c = getchar( stdin );
- D. getchar( &c )
- E. c = getchar(); [Ans]

Q.166

Code:

```
#include <stdio.h>
```

```
int i;
```

```
void increment(int i)
```

```
{
```

```
 i++;
```

```
}
```

```
int main()
```

```
{
```

```
 for(i = 0; i < 10; increment(i))
```

```
 {
```

```
 }
```

```
 printf("i=%d\n", i);
```

```
 return 0;
```

```
}
```

What will happen when the program above is compiled and executed?

- A. It will not compile.

- B. It will print out: i=9.
- C. It will print out: i=10.
- D. It will print out: i=11.
- E. It will loop indefinitely.

Q.167

Code:

```
char ptr1[] = "Hello World";
char *ptr2 = malloc(5);
ptr2 = ptr1;
```

What is wrong with the above code (assuming the call to malloc does not fail)?

- A. There will be a memory overwrite.
- B. There will be a memory leak.
- C. There will be a segmentation fault.
- D. Not enough space is allocated by the malloc.
- E. It will not compile.

Q.168

Code:

```
int i = 4;
switch (i)
{
 default:
 ;
 case 3:
 i += 5;
 if (i == 8)
 {
 i++;
 if (i == 9) break;
 i *= 2;
 }
 i -= 4;
 break;
 case 8:
 i += 5;
 break;
}
```

```
printf("i = %d\n", i);
```

What will the output of the sample code above be?

- A. i = 5
- B. i = 8
- C. i = 9

D.i = 10

E.i = 18

Q.169 Which one of the following C operators is right associative?

A.=

B.,

C.[]

D.^

E.->

Q.170

What does the "auto" specifier do?

A. It automatically initializes a variable to 0;.

B. It indicates that a variable's memory will automatically be preserved.

C. It automatically increments the variable when used.

D. It automatically initializes a variable to NULL.

E. It indicates that a variable's memory space is allocated upon entry into the block.

Q.171

How do you include a system header file called sysheader.h in a C source file?

A. #include <sysheader.h>

B. #incl "sysheader.h"

C. #includefile <sysheader>

D. #include sysheader.h

E. #incl <sysheader.h>

Q.172 Which one of the following printf() format specifiers indicates to print a double value in decimal notation, left aligned in a 30-character field, to four (4) digits of precision?

A. %-30.4e

B. %4.30e

C. %-4.30f

D. %-30.4f

E. %#30.4f

Q.173



Code:

```
int x = 0;
for (; ;)
{
 if (x++ == 4)
 break;
 continue;
}
```

printf("x=%d\n", x);

What will be printed when the sample code above is executed?

- A. x=0
- B. x=1
- C. x=4
- D. x=5
- E. x=6

Q.174

According to the Standard C specification, what are the respective minimum sizes (in bytes) of the following three data types: short; int; and long?

- A. 1, 2, 2
- B. 1, 2, 4
- C. 1, 2, 8
- D. 2, 2, 4
- E. 2, 4, 8

Q.175

Code:

```
int y[4] = {6, 7, 8, 9};
```

```
int *ptr = y + 2;
```

```
printf("%d\n", ptr[1]); /*ptr+1 == ptr[1]*/
```

What is printed when the sample code above is executed?

- A. 6
- B. 7
- C. 8
- D. 9
- E. The code will not compile.

Q.176

penny = one

nickel = five

dime = ten

quarter = twenty-five

How is enum used to define the values of the American coins listed above?

A. enum coin {(penny,1), (nickel,5), (dime,10), (quarter,25)};

B. enum coin ({penny,1}, {nickel,5}, {dime,10}, {quarter,25});

C. enum coin {penny=1,nickel=5,dime=10,quarter=25};

D. enum coin (penny=1,nickel=5,dime=10,quarter=25);

E. enum coin {penny, nickel, dime, quarter} (1, 5, 10, 25);

-

177

char txt [20] = "Hello world!\0";

How many bytes are allocated by the definition above?

A. 11 bytes

B. 12 bytes

C. 13 bytes

D. 20 bytes

E. 21 bytes

Q.178

Code:

```
int i = 4;
```

```
int x = 6;
```

```
double z;
```

```
z = x / i;
```

```
printf("z=%.2f\n", z);
```

What will print when the sample code above is executed?

A. z=0.00

B. z=1.00

C. z=1.50

D. z=2.00

E. z=NULL

179

Which one of the following variable names is NOT valid?

A. go\_cart

B. go4it

- C. 4season
- D. run4
- E. \_what

Q.180

`int a [8] = { 0, 1, 2, 3 };`

The definition of a above explicitly initializes its first four elements. Which one of the following describes how the compiler treats the remaining four elements?

- A. Standard C defines this particular behavior as implementation-dependent. The compiler writer has the freedom to decide how the remaining elements will be handled.
- B. The remaining elements are initialized to zero(0).
- C. It is illegal to initialize only a portion of the array. Either the entire array must be initialized, or no part of it may be initialized.
- D. As with an enum, the compiler assigns values to the remaining elements by counting up from the last explicitly initialized element. The final four elements will acquire the values 4, 5, 6, and 7, respectively.
- E. They are left in an uninitialized state; their values cannot be relied upon.

Q.181

Which one of the following is a true statement about pointers?

- A. They are always 32-bit values.
- B. For efficiency, pointer values are always stored in machine registers.
- C. With the exception of generic pointers, similarly typed pointers may be subtracted from each other.
- D. A pointer to one type may not be cast to a pointer to any other type.
- E. With the exception of generic pointers, similarly typed pointers may be added to each other.

Q.182

Which one of the following statements allocates enough space to hold an array of 10 integers that are initialized to 0?

- A. `int *ptr = (int *) malloc(10, sizeof(int));`
- B. `int *ptr = (int *) calloc(10, sizeof(int));`
- C. `int *ptr = (int *) malloc(10*sizeof(int));`
- D. `int *ptr = (int *) alloc(10*sizeof(int));`
- E. `int *ptr = (int *) calloc(10*sizeof(int));`

Q.183

What are two predefined FILE pointers in C?

- A. `stdout` and `stderr`
- B. `console` and `error`
- C. `stdout` and `stdio`
- D. `stdio` and `stderr`
- E. `errout` and `conout`

Q.184

Code:

```
u32 X (f32 f)
{
 union {
 f32 f;
 u32 n;
 } u;
 u.f = f;
 return u.n;
}
```

Given the function `X()`, defined above, assume that `u32` is a type-definition indicative of a 32-bit unsigned integer and that `f32` is a type-definition indicative of a 32-bit floating-point number.

Which one of the following describes the purpose of the function defined above?

- A. `X ()` effectively rounds `f` to the nearest integer value, which it returns.
- B. `X ()` effectively performs a standard typecast and converts `f` to a roughly equivalent integer.
- C. `X ()` preserves the bit-pattern of `f`, which it returns as an unsigned integer of equal size.
- D. Since `u.n` is never initialized, `X()` returns an undefined value. This function is therefore a primitive pseudorandom number generator.
- E. Since `u.n` is automatically initialized to zero (0) by the compiler, `X()` is an obtuse way of always obtaining a zero (0) value.

Q.185

Code:

```

long factorial (long x)
{
 ???
 return x * factorial(x - 1);
}

```

With what do you replace the ??? to make the function shown above return the correct answer?

- A. if (x == 0) return 0;
- B. return 1;
- C. if (x >= 2) return 2;
- D. if (x == 0) return 1;
- E. if (x <= 1) return 1;

Q.186

Code:

```

/* Increment each integer in the array 'p' of * size 'n'. */
void increment_ints (int p [/*n*/], int n)
{
 assert(p != NULL); /* Ensure that 'p' isn't a null pointer. */
 assert(n >= 0); /* Ensure that 'n' is nonnegative. */
 while (n) /* Loop over 'n' elements of 'p'. */
 {
 p++; / Increment *p. */
 p++, n--; /* Increment p, decrement n. */
 }
}

```

Consider the function `increment_ints()`, defined above. Despite its significant inline commentary, it contains an error. Which one of the following correctly assesses it?

- A. `*p++` causes `p` to be incremented before the dereference is performed, because both operators have equal precedence and are right associative.
- B. An array is a nonmodifiable lvalue, so `p` cannot be incremented directly. A navigation pointer should be used in conjunction with `p`.
- C. `*p++` causes `p` to be incremented before the dereference is performed, because the autoincrement operator has higher precedence than the indirection operator.
- D. The condition of a while loop must be a Boolean expression. The condition should be `n != 0`.

E. An array cannot be initialized to a variable size. The subscript n should be removed from the definition of the parameter p.

Q.187

How is a variable accessed from another file?

- A. The global variable is referenced via the extern specifier.
- B. The global variable is referenced via the auto specifier.
- C. The global variable is referenced via the global specifier.
- D. The global variable is referenced via the pointer specifier.
- E. The global variable is referenced via the ext specifier.

Q.188 When applied to a variable, what does the unary "&" operator yield?

- A. The variable's value
- B. The variable's binary form
- C. The variable's address
- D. The variable's format
- E. The variable's right value

Q.189

Code:

```
/* sys/cdef.h */
#if defined(__STDC__) || defined(__cplusplus)
#define __P(protos) protos
#else
#define __P(protos) ()
#endif
/* stdio.h */
#include <sys/cdefs.h>
div_t div __P((int, int));
```

The code above comes from header files for the FreeBSD implementation of the C library. What is the primary purpose of the \_\_P() macro?

- A. The \_\_P() macro has no function, and merely obfuscates library function declarations. It should be removed from further releases of the C library.
- B. The \_\_P() macro provides forward compatibility for C++ compilers, which do not recognize Standard C prototypes.

C. Identifiers that begin with two underscores are reserved for C library implementations. It is impossible to determine the purpose of the macro from the context given.

D. The `__P()` macro provides backward compatibility for K&R C compilers, which do not recognize Standard C prototypes.

E. The `__P()` macro serves primarily to differentiate library functions from application-specific functions.

Q.190 Which one of the following is NOT a valid identifier?

A. `__ident`

B. `auto`

C. `bigNumber`

D. `g42277`

E. `peaceful_in_space`

Q.191

`/* Read an arbitrarily long string. */`

Code:

```
int read_long_string (const char ** const buf) {
 char * p = NULL;
 const char * fwd = NULL;
 size_t len = 0;
 assert(buf);
 do
 {
 p = realloc(p, len += 256);
 if (!p)
 return 0;
 if (!fwd)
 fwd = p;
 else
 fwd = strchr(p, '\0');
 } while (fgets(fwd, 256, stdin));
 *buf = p;
 return 1;
}
```

The function `read_long_string()`, defined above, contains an error that may be particularly visible under heavy stress. Which one of the following

describes it?

A. The write to \*buf is blocked by the const qualifications applied to its type.

B. If the null pointer for char is not zero-valued on the host machine, the implicit comparisons to zero (0) may introduce undesired behavior. Moreover, even if successful, it introduces machine-dependent behavior and harms portability.

C. The symbol stdin may not be defined on some ANSI C compliant systems.

D. The else causes fwd to contain an errant address.

E. If the call to realloc() fails during any iteration but the first, all memory previously allocated by the loop is leaked.

Q.192

Code:

```
FILE *f = fopen(fileName, "r");
readData(f);
if(???)
{
 puts("End of file was reached");
}
```

Which one of the following can replace the ??? in the code above to determine if the end of a file has been reached?

A. f == EOF

B. feof( f )

C. eof( f )

D. f == NULL

E. !f

Q.193

Global variables that are declared static are \_\_\_\_\_.

Which one of the following correctly completes the sentence above?

A. Deprecated by Standard C

B. Internal to the current translation unit

C. Visible to all translation units

D. Read-only subsequent to initialization



## E. Allocated on the heap

Q.194

/\* Read a double value from fp. \*/

Code:

```
double read_double (FILE * fp) {
 double d;
 assert(fp != NULL);
 fscanf(fp, " %lf", d);
 return d;
}
```

The code above contains a common error. Which one of the following describes it?

- A. fscanf() will fail to match floating-point numbers not preceded by whitespace.
- B. The format specifier %lf indicates that the corresponding argument should be long double rather than double.
- C. The call to fscanf() requires a pointer as its last argument.
- D. The format specifier %lf is recognized by fprintf() but not by fscanf().
- E. d must be initialized prior to usage.

Q.195

Which one of the following is NOT a valid C identifier?

A.

\_\_\_S

B.

1\_\_\_

C.

\_\_\_1

D.

\_\_\_

E.

S\_\_\_

196

According to Standard C, what is the type of an unsuffixed floating-point literal, such as 123.45?

- A. long double
- B. Unspecified
- C. float
- D. double

E. signed float

Q.197

Which one of the following is true for identifiers that begin with an underscore?

- A. They are generally treated differently by preprocessors and compilers from other identifiers.
- B. They are case-insensitive.
- C. They are reserved for usage by standards committees, system implementers, and compiler engineers.
- D. Applications programmers are encouraged to employ them in their own code in order to mark certain symbols for internal usage.
- E. They are deprecated by Standard C and are permitted only for backward compatibility with older C libraries.

Q.198

Which one of the following is valid for opening a read-only ASCII file?

- A. `fileOpen (filenm, "r");`
- B. `fileOpen (filenm, "ra");`
- C. `fileOpen (filenm, "read");`
- D. `fopen (filenm, "read");`
- E. `fopen (filenm, "r");`

Q.199

If there is a need to see output as soon as possible, what function will force the output from the buffer into the output stream?

- A. `flush()`
- B. `output()`
- C. `fflush()`
- D. `dump()`
- E. `write()`

Q.200

`short int x; /* assume x is 16 bits in size */`

What is the maximum number that can be printed using `printf("%d\n", x)`, assuming that `x` is initialized as shown above?

- A.127
- B.128
- C.255

D.32,767

E.65,536

Q.201

Code:

```
void crash (void)
{
 printf("got here");
 *((char *) 0) = 0;
}
```

The function crash(), defined above, triggers a fault in the memory management hardware for many architectures. Which one of the following explains why "got here" may NOT be printed before the crash?

- A. The C standard says that dereferencing a null pointer causes undefined behavior. This may explain why printf() apparently fails.
- B. If the standard output stream is buffered, the library buffers may not be flushed before the crash occurs.
- C. printf() always buffers output until a newline character appears in the buffer. Since no newline was present in the format string, nothing is printed.
- D. There is insufficient information to determine why the output fails to appear. A broader context is required.
- E. printf() expects more than a single argument. Since only one argument is given, the crash may actually occur inside printf(), which explains why the string is not printed. puts() should be used instead.

Q.202

Code:

```
char * dwarves [] = {
 "Sleepy",
 "Dopey" "Doc",
 "Happy",
 "Grumpy" "Sneezy",
 "Bashful",
};
```

How many elements does the array dwarves (declared above) contain?

Assume the C compiler employed strictly complies with the requirements of Standard C.

A.4

B.5

C.6

D.7  
E.8

Q.203

Which one of the following can be used to test arrays of primitive quantities for strict equality under Standard C?

- A. qsort()
- B. bcmp()
- C. memcmp()
- D. strxfrm()
- E. bsearch()

204

Code:

```
int debug (const char * fmt, ...) {
 extern FILE * logfile;
 va_list args;
 assert(fmt);
 args = va_arg(fmt, va_list);
 return vfprintf(logfile, fmt, args);
}
```

The function debug(), defined above, contains an error. Which one of the following describes it?

- A. The ellipsis is a throwback from K&R C. In accordance with Standard C, the declaration of args should be moved into the parameter list, and the K&R C macro va\_arg() should be deleted from the code.
- B. vfprintf() does not conform to ISO 9899: 1990, and may not be portable.
- C. Library routines that accept argument lists cause a fault on receipt of an empty list. The argument list must be validated with va\_null() before invoking vfprintf().
- D. The argument list args has been improperly initialized.
- E. Variadic functions are discontinued by Standard C; they are legacy constructs from K&R C, and no longer compile under modern compilers.

Q.205

Code:

```
char *buffer = "0123456789";
char *ptr = buffer;
ptr += 5;
printf("%s\n", ptr);
```

```
printf("%s\n", buffer);
```

What will be printed when the sample code above is executed?

A.

0123456789

56789

B.

5123456789

5123456789

C.

56789

56789

D.

0123456789

0123456789

E.

56789

0123456789

Q.206

/\* Get the rightmost path element of a Unix path. \*/

Code:

```
char * get_rightmost (const char * d)
{
 char rightmost [MAXPATHLEN];
 const char * p = d;
 assert(d != NULL);
 while (*d != '\0')
 {
 if (*d == '/')
 p = (*(d + 1) != '\0') ? d + 1 : p;
 d++;
 }
 memset(rightmost, 0, sizeof(rightmost));
 memcpy(rightmost, p, strlen(p) + 1);
 return rightmost;
}
```

The function `get_rightmost()`, defined above, contains an error. Which one of the following describes it?

A. The calls to `memset()` and `memcpy()` illegally perform a pointer conversion on `rightmost` without an appropriate cast.

- B. The code does not correctly handle the situation where a directory separator '/' is the final character.
- C. The if condition contains an incorrectly terminated character literal.
- D. memcpy() cannot be used safely to copy string data.
- E. The return value of get\_rightmost() will be invalid in the caller's context.

Q.207 What number is equivalent to -4e3?

- A.-4000
- B.-400
- C.-40
- D..004
- E..0004

Q.208

Code:

```
void freeList(struct node *n)
{
 while(n)
 {
 ???
 }
}
```

Which one of the following can replace the ??? for the function above to release the memory allocated to a linked list?

- A.  
n = n->next;  
free( n );
- B.  
struct node m = n;  
n = n->next;  
free( m );
- C.  
struct node m = n;  
free( n );  
n = m->next;

D.

```
free(n);
n = n->next;
```

E.

```
struct node m = n;
free(m);
n = n->next;
```

Q.209

How does variable definition differ from variable declaration?

- A. Definition allocates storage for a variable, but declaration only informs the compiler as to the variable's type.
- B. Declaration allocates storage for a variable, but definition only informs the compiler as to the variable's type.
- C. Variables may be defined many times, but may be declared only once.
- D. Variable definition must precede variable declaration.
- E. There is no difference in C between variable declaration and variable definition.

Q.210

Code:

```
int x[] = {1, 2, 3, 4, 5};
int u;
int *ptr = x;
????
for(u = 0; u < 5; u++)
{
 printf("%d-", x[u]);
}
printf("\n");
```

Which one of the following statements could replace the ??? in the code above to cause the string 1-2-3-10-5- to be printed when the code is executed?

- A. \*ptr + 3 = 10;
- B. \*ptr[ 3 ] = 10;
- C. \*(ptr + 3) = 10;
- D. (\*ptr)[ 3 ] = 10;
- E. \*(ptr[ 3 ]) = 10;

Q.211

Code:

```
/*sum_array() has been ported from FORTRAN. No * logical changes have
been made*/
```

```
double sum_array(double d[],int n)
{
 register int i;
 double total=0;
 assert(d!=NULL);
 for(i=1;i<=n;i++)
 total+=d[i];
 return total;
}
```

The function sum\_array(), defined above, contains an error. Which one of the following accurately describes it?

- A. The range of the loop does not match the bounds of the array d.
- B. The loop processes the incorrect number of elements.
- C. total is initialized with an integer literal. The two are not compatible in an assignment.
- D. The code above fails to compile if there are no registers available for i.
- E. The formal parameter d should be declared as double \* d to allow dynamically allocated arrays as arguments.

Q.212

Code:

```
#include <stdio.h>
```

```
void func()
```

```
{
 int x = 0;
 static int y = 0;
 x++; y++;
 printf("%d -- %d\n", x, y);
}
```

```
int main()
```

```
{
 func();
 func();
 return 0;
}
```

What will the code above print when it is executed?

- A. 1 -- 1  
1 -- 1
- B. 1 -- 1  
2 -- 1
- C. 1 -- 1  
2 -- 2



D.1 -- 0  
1 -- 0  
E.1 -- 1  
1 -- 2

Q.213

Code:

```
int fibonacci (int n)
{
 switch (n)
 {
 default:
 return (fibonacci(n - 1) + fibonacci(n - 2));
 case 1:
 case 2:
 }
 return 1;
}
```

The function above has a flaw that may result in a serious error during some invocations. Which one of the following describes the deficiency illustrated above?

- A. For some values of n, the environment will almost certainly exhaust its stack space before the calculation completes.
- B. An error in the algorithm causes unbounded recursion for all values of n.
- C. A break statement should be inserted after each case. Fall-through is not desirable here.
- D. The fibonacci() function includes calls to itself. This is not directly supported by Standard C due to its unreliability.
- E. Since the default case is given first, it will be executed before any case matching n.

Q.214

When applied to a variable, what does the unary "&" operator yield?

- A. The variable's address
- B. The variable's right value
- C. The variable's binary form
- D. The variable's value
- E. The variable's format

Q.215

```
short int x; /* assume x is 16 bits in size */
```

What is the maximum number that can be printed using `printf("%d\n", x)`, assuming that `x` is initialized as shown above?

A.127

B.128

C.255

D.32,767

E.65,536

Q.216

```
int x = 011 | 0x10;
```

What value will `x` contain in the sample code above?

A.3

B.13

C.19

D.25

E.27

Q. 217

Which one of the following calls will open the file `test.txt` for reading by `fgetc`?

A. `fopen( "test.txt", "r" );`

B. `read( "test.txt" )`

C. `fileopen( "test.txt", "r" );`

D. `fread( "test.txt" )`

E. `freopen( "test.txt" )`

Q.218

Code:

```
int m = -14;
```

```
int n = 6;
```

```
int o;
```

```
o = m % ++n;
```

```
n += m++ - o;
```

```
m <=<= (o ^ n) & 3;
```

Assuming two's-complement arithmetic, which one of the following correctly represents the values of `m`, `n`, and `o` after the execution of the code above?

- A. m = -26, n = -7, o = 0
- B. m = -52, n = -4, o = -2
- C. m = -26, n = -5, o = -2
- D. m = -104, n = -7, o = 0
- E. m = -52, n = -6, o = 0

Q.219

How do printf()'s format specifiers %e and %f differ in their treatment of floating-point numbers?

- A. %e displays an argument of type double with trailing zeros; %f never displays trailing zeros.
- B. %e displays a double in engineering notation if the number is very small or very large. Otherwise, it behaves like %f and displays the number in decimal notation.
- C. %e always displays an argument of type double in engineering notation; %f always displays an argument of type double in decimal notation. [Ans]
- D. %e expects a corresponding argument of type double; %f expects a corresponding argument of type float.
- E. %e and %f both expect a corresponding argument of type double and format it identically. %e is left over from K&R C; Standard C prefers %f for new code.

Q.220

```
c = getchar();
```

What is the proper declaration for the variable c in the code above?

- A. char \*c;
- B. unsigned int c;
- C. int c;
- D. unsigned char c;
- E. char c;

Q.223 Which one of the following will define a function that CANNOT be called from another source file?

- A. void function() { ... }
- B. extern void function() { ... }
- C. const void function() { ... }
- D. private void function() { ... }